Northeastern University

(Computer Science)

Team Indigenous Tech Circle

Green Software Engineering

Author: **Liyao Zhang**
ID: 002837359
E-mail:
zhang.liya@northeastern.edu

Author: **Wei Zhao**
ID: 001969892
E-mail:
zhao.wei2@northeastern.edu

Author: **Jun Liu**
ID: 001611944
E-mail: *liu.jun5@northeastern.edu*

Author: **Ruijin Zhang**
ID: 002622440
E-mail:
zhang.ruijin@northeastern.edu

Tutor: **Yvonne Coady**
Department: Computer Science
E-mail: *m.coady@northeastern.edu*

December 4, 2024

Contents

1 Problem Statement

As the Information and Communication Technology (ICT) sector rapidly expands, the energy consumption and greenhouse gas emissions from software development have become a global concern. However, existing frameworks and development practices lack the tools or standardized processes to effectively measure and improve software energy efficiency. Green software engineering addresses this issue by proposing design principles and practices that reduce environmental impact while improving energy efficiency. Despite this, many in the industry face challenges in applying these green principles due to a lack of real-world examples and measurable outcomes.

This project aims to provide a comprehensive framework for evaluating and improving software energy efficiency through green software engineering principles and practices. To ground this framework in real-world application, we have chosen Indigenous Tech Circle (ITC) as a case study. In recent years, although federal, provincial, and municipal governments, along with large corporations, have set procurement targets to support Indigenous capacity development, Indigenous-led companies often find themselves at a disadvantage. Larger companies, well-versed in navigating the complex procurement processes, know how to accumulate points in the bidding process, increasing their chances of success. This results in Indigenous-led companies struggling to scale or win larger contracts.

In response to this challenge, this project will analyze the current procurement landscape, providing insights for Indigenous-led companies and identifying unique opportunities they can exploit. By examining procurement opportunities, the project will not only offer strategies for Indigenous companies to gain competitive advantages but also demonstrate how green software design can be practically implemented to enhance energy efficiency and reduce environmental impact. This will help Indigenous companies achieve sustainable growth and improve their position in the competitive procurement market.

By focusing on the ITC case study, we will showcase how green software design can be applied in practice and explore its potential in reducing greenhouse gas emissions within the ICT sector. Ultimately, the goal of this project is to provide a comprehensive solution for Indigenous companies, helping them succeed in the competitive procurement market while contributing to global sustainability efforts.

2 Proposed solution

Our solution focuses on designing a platform that prioritizes energy efficiency by making several key design choices at different stages of the system architecture, including data storage, and front-end optimizations. Related work on green software engineering either focus on best practices but without quantifying their impact on a holistic basis or propose an evaluation methodology without concrete examples from real-world applications.

By drawing from the lessons learned in the literature, our solution incorporates green software principles while providing a practical example of implementation in a real-world context. We compare and contrast two distinct system design architectures to explore and discuss the following: first, the options to consider for creating a greener system design in real-world scenarios; second, methods for quantifying the impact of greener implementations using established evaluation techniques; and finally, the specific components that may significantly influence energy consumption.

2.1 Literature Review

Previous research indicates that while practitioners are aware of and care about energy efficiency in software engineering, they often face significant barriers due to a lack of comprehensive information and tools available. Manotas et al. conducted an empirical study that highlights this challenge, revealing that developers need support to incorporate such practices into their workflows [1]. Mourão et al. conducted a systematic mapping study that categorizes and synthesizes various researches concerning sustainable software engineering, emphasizing the need for more validation studies focusing on the impacts of sustainability in the future [2]. Brunnert proposed specific green software metrics, resource demand, as the basis to help developers allocate the energy consumption of the entire server to individual software components, yet no substantial prototype has been implemented to finalize the carbon emission calculation [3]. As web technologies continue to thrive and to be adopted, various resources are available to the community, aiming to bring awareness regarding sustainable software engineering [4]. Meanwhile, best practices identified in mobile application development adapting energy constraints on mobile devices can be effectively translated to web development. Rani et al. explored 20 mobile energy patterns that could be adapted for web and interviewed several expert web developers to challenge them. Finally, two of the ported patterns were evaluated against their antipatterns in terms of energy consumption [5].

A more detailed analysis focusing on architectural design was also related to the domain of mobile development, where Singh proposed a novel RMVRVM paradigm to remove the heavylifting to the back-end to reduce unnecessary data transferred to the client devices, resulting in significant decrease in battery consumption and improvement in response time [6].

The ITC project addresses the challenge of managing vast datasets in ways that align with the sustainability goals of Indigenous communities. This management involves handling large volumes of data, closing gaps, accurately categorizing information, and identifying significant feature relationships. Effective data processing is crucial for deriving actionable insights that facilitate informed decision-making. To minimize its environmental impact, the ITC project employs a green data analysis methodology aimed at reducing waste and conserving resources, essential for lowering the ecological footprint of its data management activities.In evaluating the greenness of their data solutions, the ITC project utilizes several key metrics: Energy efficiency involves measuring the power consumption of servers, storage, and network equipment; resource utilization assesses the effectiveness of the use of computational and storage resources. The carbon footprint is estimated by calculating the total greenhouse gas emissions from the energy consumed by the IT infrastructure. Furthermore, a lifecycle assessment provides a detailed examination of the environmental impacts at each stage of the data processing solution, offering a comprehensive perspective on sustainability [7].

2.2 Architecture Design

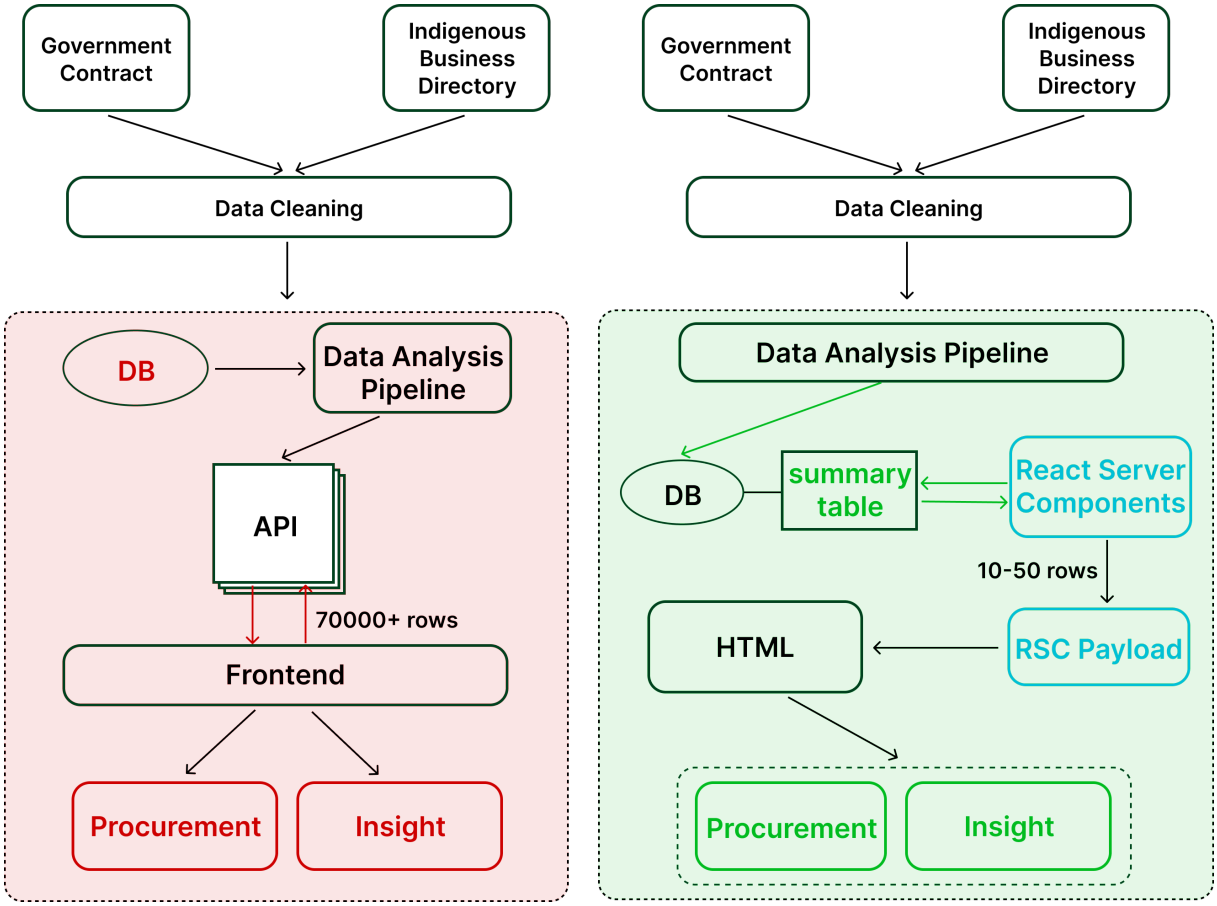


Figure 1: Two possible system architecture diagrams

2.2.1 Data Collection

Instead of relying on real-time updates, our system will primarily conduct historical retrospective analysis. This shift is driven by the fact that the procurement data we are handling is not updated frequently. Real-time data collection can be resource-intensive, especially in terms of energy consumption and network usage.

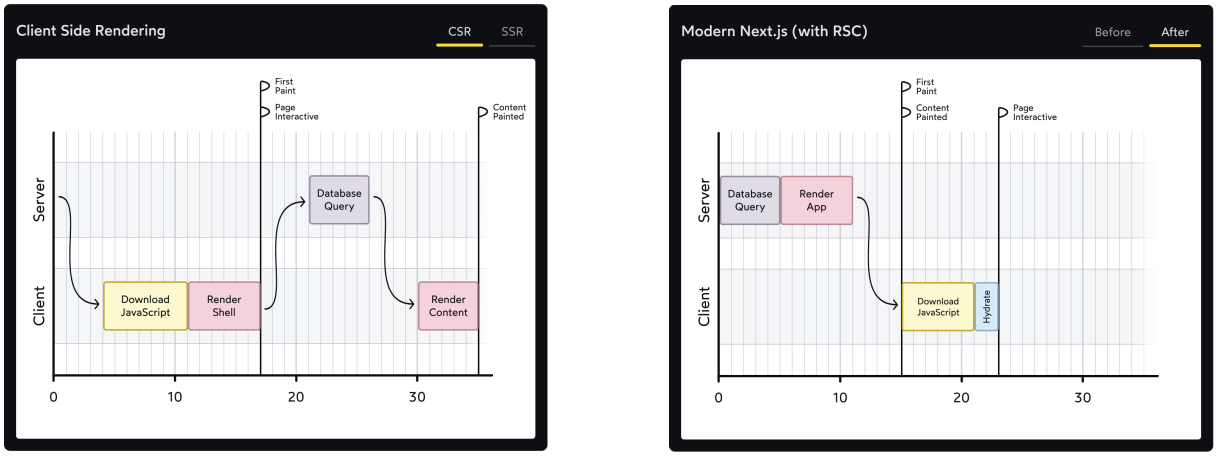
2.2.2 Data Storage

The cleaned data will go through a data analysis pipeline and will be stored in a database. However, the type of database and the order of the process may vary. An OLAP data warehouse is preferred over a traditional relational schema because the warehouse promotes summary tables that store computed results which can be efficiently retrieved to generate charts and diagrams. Therefore, the insight section will load faster under this architecture. Moreover, all the new data is supposed to go through the pipeline only once.

2.2.3 Front-end Optimization

Inspired by the RMVRVM paradigm that moves heavylifting to the server [6], we decided to embrace the most cutting-edge Server-Side Rendering paradigm leveraging React Server

Components that essentially achieves the same objective. Specifically, we will be using Next.js 14 App Router, the most popular full-stack web framework that has been adopted by the industry with all the benefits of React Server Components built in. Compared to the traditional mental model that the client is responsible for initiating the data fetching, the new pattern could get rid of API endpoints once for all because the data will be fetched on Server Components instead, which will eventually be rendered as HTML and reconcile with the static content displayed on our pages through the process called "hydration". In this way, the JavaScript bundle size and number of network round-trips between server and client are minimized, enhancing not only the responsiveness but also the security of our application [8]. On the granular level, while there are overwhelming amount of tips and best practices regarding sustainable web development, we decided to focus on one major aspect: allocate resources based on user needs and reduce unnecessary data flow resulted. For example, server-side pagination will be implemented, limiting the initial amount of data transferred from the server to 10-50 rows instead of potentially more than 70000+ rows.



(a) Client side rendering

(b) Server side rendering with RSC

Figure 2: Comparison on key metrics of web page performance between Client-side Rendering and Server-side Rendering[8]

3 Evaluation Methodology

Our evaluation of software energy and resource efficiency is based on the Green Software Measurement Model (GSMM)[9]. This comprehensive tool enables structured analysis of energy consumption in software, focusing on specific components such as CPU, memory, network, and energy usage. In addition to energy and resource efficiency, we incorporate user-centric performance metrics to ensure an optimal balance between energy sustainability and user experience.

3.1 Key Terms

- **Idle State:** In this scenario, the system is active but no user tasks are being performed. This allows us to record the baseline energy consumption and track

hardware resource usage, such as CPU and memory, when the system is not actively processing tasks.

- **Task Execution:** In this scenario, we execute typical tasks or workloads that the software is expected to handle. Performance is evaluated using both energy consumption data via GreenFrame and user-centric performance metrics:
  - **First Contentful Paint (FCP):** measures the time from when the page starts loading to when any part of the page's content is rendered on the screen.
  - **Largest Contentful Paint (LCP):** measures the time from when the page starts loading to when the largest text block or image element is rendered on the screen.
  - **Interaction to Next Paint (INP):** measures the latency of every tap, click, or keyboard interaction made with the page, and—based on the number of interactions—selects the worst interaction latency of the page (or close to the highest) as a single, representative value to describe a page's overall responsiveness.
  - **Total Blocking Time (TBT):** measures the total amount of time between FCP and TTI where the main thread was blocked for long enough to prevent input responsiveness.
  - **Cumulative Layout Shift (CLS):** measures the cumulative score of all unexpected layout shifts that occur between when the page starts loading and when its lifecycle state changes to hidden.
  - **Time to First Byte (TTFB):** measures the time it takes for the network to respond to a user request with the first byte of a resource.

  These metrics are collected in conjunction with energy consumption, CPU utilization, memory usage, and network traffic to provide insights into both the user experience and energy efficiency of the software during standard operations.

3.2 Data sources

The data for our simulations are obtained through GreenFrame, which provides real-time insights into hardware resource usage (CPU, memory, network, energy). Additionally, tools like Lighthouse and WebPageTest are employed to track user-centric performance metrics. These metrics are combined with energy data from GreenFrame to assess both resource efficiency and user experience.

3.3 Literature Review

The foundation of our evaluation methodology is derived from the Green Software Measurement Model (GSMM)[9], which synthesizes several established approaches for assessing the energy consumption of software.

In addition to energy metrics, we have incorporated user-centric performance metrics[10] to ensure that while optimizing for energy efficiency, we do not compromise user experience. The key metrics we reference are outlined in the article User-Centric Performance Metrics. This resource emphasizes metrics like First Contentful Paint (FCP),

Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS), and Total Blocking Time (TBT), all of which provide a comprehensive view of how users perceive performance during page load. By integrating these user experience metrics with the energy-focused analysis provided by GreenFrame, we ensure a holistic approach to performance optimization that balances sustainability with usability.

The User-Centric Performance Metrics article offers a well-defined framework for measuring perceived load times and stability of content, which are critical to evaluating how performance improvements are experienced by the end user. These metrics help bridge the gap between technical energy efficiency and real-world user interaction, making them a valuable addition to our evaluation process.

By combining insights from Green Software Measurement Model (GSMM), GreenFrame and user-centric performance metrics, we achieve a dual focus on both energy sustainability and the user's experience, offering a more comprehensive approach to evaluating modern web applications.

3.4 Plan of Evaluation

Our evaluation will involve both framework and component-level comparison as outlined in the architecture design for two main pages in our application: Procurement and Insight.

The Procurement page is essentially rendering a data table of procurements fetched from database. Therefore, a framework-level comparison between Next.js and Express server + React frontend as well as a component-level comparison on whether server-side pagination is implemented will be conducted. The Insight Page will render 12 insight charts calculated and sourced from the same data source. Similarly, the framework-level comparison is in place, and the summary tables will be created to basically cache the calculation results that are necessary for generating those charts. However, since the energy saved by the summary tables can only be measured on the server, our current tools that capture activities on the browser don't take them into account.

For each test, we will run the same GreenFrame scenarios and generate Lighthouse report. Specifically, the scenarios of Procurement page are initial loading of the data, filtering by vendor name, sorting by contract value, then click next page for 5 times; the scenarios of Insight page are simply loading the charts and then scroll to the footer. The Lighthouse reports reflect the performance in terms of speed of initial loading for each page and provide user-centric performance metrics.

4 Results

4.1 Lighthouse Report

As the representation of users' perception, Lighthouse report includes user-centric performance metrics such as First Contentful Paint (FCP), Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS), and Total Blocking Time (TBT). Generally, the lower those values are, the more responsive the web page is. Figure 3 highlights the performance boost gained within Next.js by adopting server-side pagination, as well as the performance boost gained by using Next.js server side rendering compared to traditional Client-API-Server architecture.

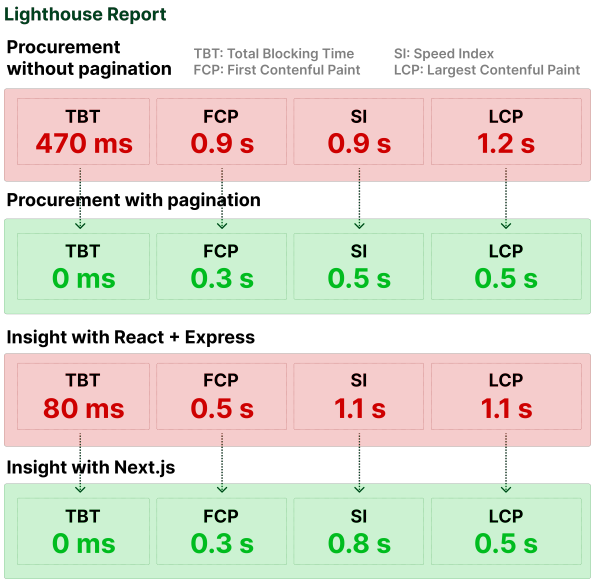


Figure 3: Lighthouse Report

4.2 GreenFrame Analysis

The GreenFrame software leverages Docker container to measure the resources usage such as CPU, Network, Screen, Memory in the GreenFrame browser. In our case, we used URLs of our deployed web applications to run the scenarios. Since we are using the same scenarios as mentioned before, we don't expect the total memory usage to be any different, which is indeed the case.

However, to our surprise, the overall CPU and Network usage of our Next.js version pages are slightly more than that of Express+React pages. If we take a closer look at the Network change over time as Figure 4 illustrates, the peak bandwidth of the Next.js page is almost half of the Express+React page, and the Network activities start immediately while there is one-second delay under the Express+React architecture. This suggests that while Next.js applications can feel more responsive but there is some additional overhead in the process, which is a trade-off between user experience and energy consumption. Meanwhile, the peak Network usage is much lower thanks to the Streaming feature of Next.js, which prevents slow data requests from blocking the whole page. This allows the user to see and interact with parts of the page without waiting for all the data to load before any UI can be shown to the user.

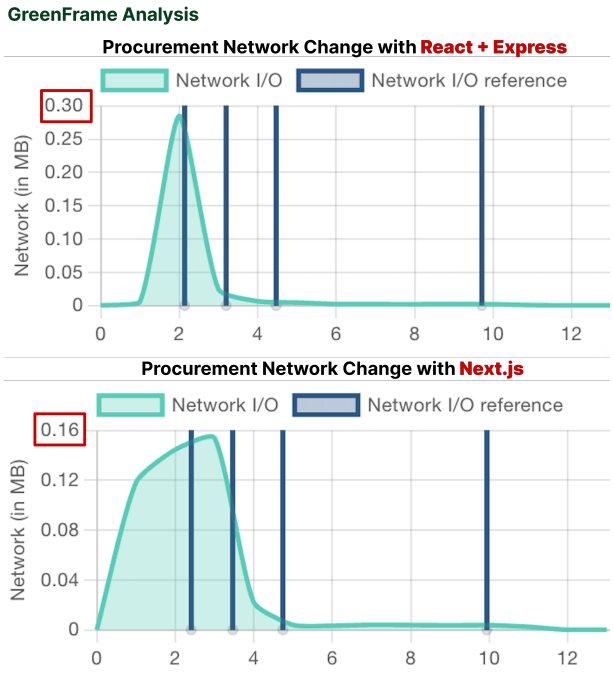


Figure 4: GreenFrame Analysis

4.3 Conclusion

Summary

Procurement	CPU in mWh	Network in mWh	Memory in GB	FCP in s	LCP in s	TBT in ms
React + Express + pagination	3	3.7	0.4	0.5	0.5	0
Next.js + pagination	4.3	5.1	0.41	0.3	0.5	0
Next.js	3.9	18	0.51	0.9	1.2	470

Insight	CPU in mWh	Network in mWh	Memory in GB	FCP in s	LCP in s	TBT in ms
React + Express	2.6	3.5	0.18	0.5	1.1	80
Next.js	2.8	4.3	0.18	0.3	0.5	0

Figure 5: Summary

Unoptimized pages show severe performance bottlenecks, while introducing pagination and using Next.js significantly improve loading speed and user experience, showcasing the benefits of modern frameworks and rendering strategies.

The procurement page without pagination suffers from significant bottlenecks, with a TBT of 470ms and FCP, SI, and LCP exceeding 0.9 seconds, largely due to excessive data loads and a lack of optimization, resulting in slow responsiveness and poor user experience.

Introducing pagination addresses these issues, significantly improving performance, as all metrics reach optimal levels: TBT drops to 0ms, FCP to 0.3 seconds, and SI and LCP to 0.5 seconds each, demonstrating reduced data transfer and computational load. On the Insight page, the React + Express implementation offers moderate improvements, with TBT at 80ms and FCP at 0.5 seconds; however, SI and LCP remain at 1.1 seconds, reflecting inefficiencies in handling complex rendering tasks. In contrast, the Next.js implementation achieves optimal performance, with TBT at 0ms, FCP at 0.3 seconds, SI at 0.8 seconds, and LCP at 0.5 seconds, showcasing the benefits of server-side rendering in optimizing resource utilization and enhancing loading speed.

To conclude, Next.js with RSCs improve the responsiveness of web pages, especially during the initial load, but with a slight increase in the overall CPU and Network usages. Meanwhile, best practices such as server-side pagination remain essential to avoid significant network overhead when handling large datasets.

5 Future work

Our project has explored the principles of green software engineering and demonstrated its practical application through the ITC case study. However, green software engineering is a continually evolving field, and future work will expand and deepen the research in the following directions:

First, we are currently evaluating only the frontend energy consumption, and the impact made by summary tables is not fully captured. Second, we are mainly using the same set of tools for testing purposes, which may cause biases. In the future, we could test with other similar tools to further validate our findings. Third, the reason why React Server Components contributed to more energy consumption is not fully investigated due to the time limit. One possible reason is that while the Next.js paradigm reduces the JavaScript bundle size, the payload size for each network request will be larger afterwards, leading to more network usage over time.

References

[1] I. Manotas et al. "An Empirical Study of Practitioners' Perspectives on Green Software Engineering". In: *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. New York, NY, USA: Association for Computing Machinery, May 2016, pp. 237–248. DOI: 10.1145/2884781.2884810.

[2] B. C. Mourão, L. Karita, and I. do Carmo Machado. "Green and Sustainable Software Engineering - A Systematic Mapping Study". In: *Proceedings of the XVII Brazilian Symposium on Software Quality (SBQS '18)*. New York, NY, USA: Association for Computing Machinery, Oct. 2018, pp. 121–130. DOI: 10.1145/3275245.3275258.

[3] A. Brunnert. "Green Software Metrics". In: *Companion of the 15th ACM/SPEC International Conference on Performance Engineering (ICPE '24 Companion)*. New York, NY, USA: Association for Computing Machinery, May 2024, pp. 287–288. DOI: 10.1145/3629527.3652883.

[4] R. D. Caballar. "We Need to Decarbonize Software". In: *IEEE Spectrum* (Mar. 2024). Retrieved September 15, 2024. URL: https://spectrum.ieee.org/green-software.

[5] P. Rani et al. "Energy Patterns for Web: An Exploratory Study". In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*. Lisbon, Portugal: ACM, Apr. 2024, pp. 12–22. DOI: 10.1145/3639475.3640110.

[6] L. Singh. "RMVRVM – A Paradigm for Creating Energy Efficient User Applications Connected to Cloud through REST API". In: *Proceedings of the 15th Innovations in Software Engineering Conference (ISEC '22)*. New York, NY, USA: Association for Computing Machinery, Feb. 2022, pp. 1–11. DOI: 10.1145/3511430.3511434.

[7] S. R. A. Ibrahim et al. "The Development of Green Software Process Model". In: *International Journal of Advanced Computer Science and Applications* (2021). DOI: 10.14569/ijacsa.2021.0120869.

[8] J. W. Comeau. *Making Sense of React Server Components*. https://www.joshwcomeau.com/react/server-components/. Accessed: Oct. 22, 2024.

[9] Achim Guldner et al. "Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green Software Measurement Model (GSMM)". In: *Future Gener. Comput. Syst.* 155.C (July 2024), pp. 402–418. ISSN: 0167-739X. DOI: 10.1016/j.future.2024.01.033. URL: https://doi.org/10.1016/j.future.2024.01.033.

[10] Philip Walton. "User-Centric Performance Metrics | Articles". In: *web.dev* (2023). URL: https://web.dev/articles/user-centric-performance-metrics.